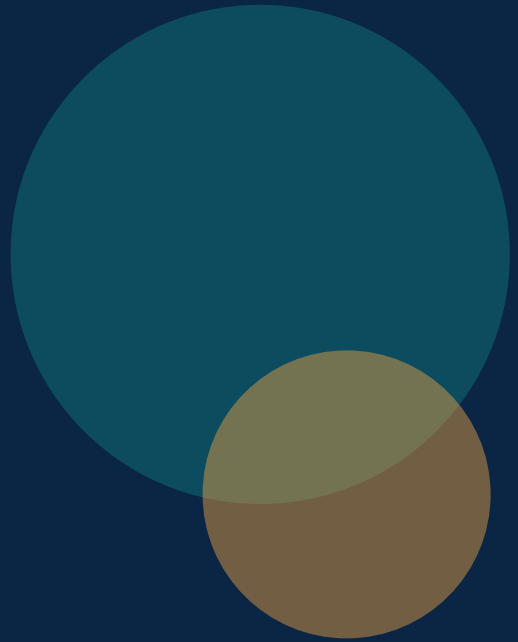
TP 1

Python pour la science des données

Jupyter, NumPy, Pandas, visualisation et premier modèle de Machine Learning



Objectifs et plan du TP

Partie 1

Jupyter et bases de Python

- Lancer Jupyter, comprendre un notebook
- Variables, types, listes, dictionnaires
- Conditions et boucles
- Fonctions et import de bibliothèques

Partie 2

Manipulation de données

- NumPy : tableaux, indexation, opérations, agrégations
- Pandas : CSV, sélection, filtre, tri, groupby
- Valeurs manquantes
- NumPy ou Pandas : que choisir ?

Partie 3

Visualisation et premier ML

- Matplotlib et Seaborn : courbes, histogrammes, nuages, corrélations
- Scikit-learn : jeu Iris, features et target
- Séparation train/test
- Premier modèle simple

NumPy : créer des tableaux

NumPy fournit le tableau ndarray : une grille de nombres de même type, rapide et pratique pour le calcul.

Python

```python
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.arange(0, 10, 2)
d = np.linspace(0, 1, 5)
z = np.zeros((2, 3))
m = np.arange(6).reshape(2, 3)

print(a, b)
print(d)
print(z)
print(m, m.shape)
```

Résultat

```
[1 2 3 4] [0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
[[0. 0. 0.]
 [0. 0. 0.]]
[[0 1 2]
 [3 4 5]] (2, 3)
```

À retenir

- array : à partir d'une liste ; arange : suite avec un pas
- linspace(0, 1, 5) : 5 points régulièrement espacés
- reshape(2, 3) : 2 lignes, 3 colonnes ; shape donne les dimensions

NumPy : indexer et filtrer

On accède aux éléments avec [ligne, colonne]. Les deux-points sélectionnent « tout » ou une tranche.

Python

```python
m = np.array([[10, 20, 30],
              [40, 50, 60]])
print(m[0, 1])
print(m[:, 2])
print(m[1, :2])
mask = m > 25
print(mask)
print(m[mask])
```

Résultat

```
20
[30 60]
[40 50]
[[False False  True]
 [ True  True  True]]
[30 40 50 60]
```

À retenir

- m[0, 1] : ligne 0, colonne 1
- m[:, 2] : toutes les lignes, colonne 2
- m > 25 donne un masque booléen ; m[masque] garde les True

NumPy : opérations et broadcasting

Les opérations s'appliquent à tout le tableau d'un coup (vectorisation) : pas besoin de boucle for.

Python

```python
a = np.array([1, 2, 3])
print(a * 2)
print(a + a)
print(a ** 2)
col = np.array([[10], [20]])
print(a + col)
```

Résultat

```
[2 4 6]
[2 4 6]
[1 4 9]
[[11 12 13]
 [21 22 23]]
```

Pourquoi c'est important ?

Une opération vectorisée est écrite en une ligne et s'exécute bien plus vite qu'une boucle Python sur 1 million de valeurs.

À retenir

- a * 2 multiplie chaque élément
- Broadcasting : NumPy étire le plus petit tableau pour l'adapter au plus grand
- Ici, col (2×1) et a (3,) forment un résultat 2×3

NumPy : agrégations

Une agrégation résume un tableau : somme, moyenne, maximum… Le paramètre axis choisit la direction.

Python

```python
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.sum(), m.mean())
print(m.sum(axis=0))
print(m.sum(axis=1))
print(m.max(), m.argmax())
```

Résultat

```
21 3.5
[5 7 9]
[ 6 15]
6 5
```

À retenir

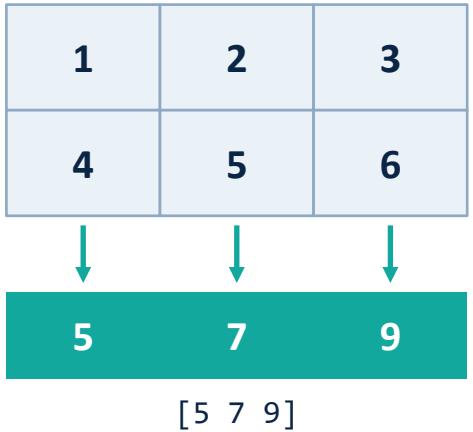
- Sans axis : calcul sur tous les éléments
- axis=0 : résultat par colonne ; axis=1 : résultat par ligne
- argmax donne la position du maximum (**ici 5, tableau aplati**)

Comprendre axis

axis indique la dimension qui disparaît après le calcul. Matrice utilisée : m = [[1, 2, 3], [4, 5, 6]].

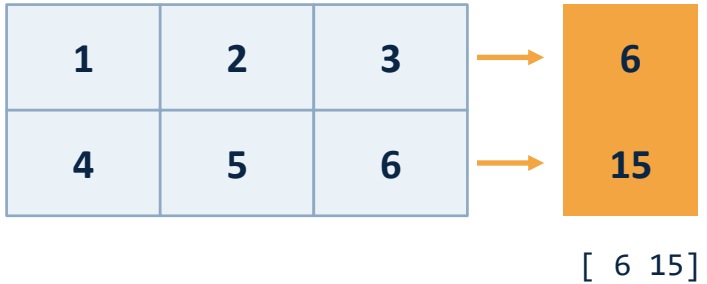
`m.sum(axis=0)`

On additionne vers le bas : une valeur par colonne



`m.sum(axis=1)`

On additionne vers la droite : une valeur par ligne



Moyen mnémotechnique : axis=0 « écrase » les lignes, axis=1 « écrase » les colonnes. Le résultat perd la dimension indiquée.

Pandas : charger un fichier CSV

Pandas lit un CSV et le range dans un DataFrame : un tableau avec des noms de colonnes et un index de lignes.

```python
import pandas as pd

df =pd.read_csv("etudiants.csv")
print(df.head(3))
print(df.shape)
print(df.dtypes)
```

Résultat

```
      nom        filiere  age  note_maths  note_info
0   Amine   Informatique   20        14.5       16.0
1    Sara  Mathématiques   21        17.0       15.5
2  Yacine   Informatique   19        11.0        NaN
(10, 5)
nom               str
filiere           str
age             int64
note_maths    float64
note_info     float64
dtype: object
```

Contenu du fichier etudiants.csv (début)

```
nom,filiere,age,note_maths,note_info
Amine,Informatique,20,14.5,16.0
Sara,Mathématiques,21,17.0,15.5
Yacine,Informatique,19,11.0,
```

Une case vide dans le CSV devient NaN (valeur manquante).

À retenir

- read_csv charge le fichier ; head(3) : 3 premières lignes
- shape : (lignes, colonnes) ; dtypes : types (str peut s'afficher object)

Pandas : sélectionner

On choisit des colonnes par leur nom, et des lignes ou cellules par étiquette (loc) ou par position (iloc).

Python

```python
print(df["nom"].head(3))
print(df[["nom", "age"]].head(3))
print(df.loc[0, "nom"])
print(df.iloc[0:2, 0:2])
```

À retenir

- df["nom"] : une colonne (Series)
- df[["nom", "age"]] : plusieurs colonnes (DataFrame)
- loc[0, "nom"] : étiquettes ; iloc[0:2, 0:2] : positions

Résultat

```
0     Amine
1      Sara
2    Yacine
Name: nom, dtype: str
      nom  age
0   Amine   20
1    Sara   21
2  Yacine   19
Amine
     nom        filiere
0  Amine   Informatique
1   Sara  Mathématiques
```

Pandas : filtrer et trier

On filtre avec une condition booléenne et on trie avec sort_values.

Python

```python
info = df[df["filiere"] == "Informatique"]
print(info[["nom", "note_maths"]])

bons = df[(df["note_maths"] >= 14)
          & (df["age"] <= 21)]
print(bons["nom"].tolist())

top = df.sort_values("note_maths",
                     ascending=False)
print(top[["nom", "note_maths"]].head(3))
```

Résultat

```
      nom  note_maths
0   Amine        14.5
2   Yacine       11.0
5   Nadia        15.0
8  Sofiane       12.0
['Amine', 'Sara', 'Nadia']
     nom  note_maths
1   Sara        17.0
7  Meriem       16.5
5   Nadia       15.0
```

À retenir

- Plusieurs conditions : & (et), | (ou), avec des parenthèses
- Ne pas écrire and / or : ils ne marchent pas ici
- ascending=False : tri décroissant

Pandas : groupby

groupby regroupe les lignes par valeur d'une colonne, puis calcule un résumé pour chaque groupe.

Python

```python
g = df.groupby("filiere")
print(g["note_maths"].mean())

stats = g.agg(
    effectif=("nom", "count"),
    moy_info=("note_info", "mean"))
print(stats)
```

Résultat

```
filiere
Informatique     13.125000
Mathématiques    15.666667
Physique         10.000000
Name: note_maths, dtype: float64
               effectif   moy_info
filiere
Informatique          4  15.500000
Mathématiques         3  14.750000
Physique              3  11.333333
```

À retenir

- g["note_maths"].mean() : moyenne par filière
- agg : plusieurs résumés à la fois, avec noms choisis
- Les NaN sont ignorés dans les moyennes

groupby en trois temps

Principe « séparer, appliquer, combiner » sur 6 lignes de l'exemple : moyenne de note_maths par filière.

1. Données

nom	filiere	maths
Amine	Info	14.5
Sara	Maths	17.0
Yacine	Info	11.0
Lina	Phys	NaN
Karim	Maths	13.5
Omar	Phys	9.5

2. Séparer, puis calculer

Info : 14.5 ; 11.0
moyenne = 12.75

Maths : 17.0 ; 13.5
moyenne = 15.25

Phys : NaN ; 9.5
moyenne = 9.5 (NaN ignoré)

3. Combiner

filiere	maths
Info	12.75
Maths	15.25
Phys	9.5

```
df.groupby("filiere")["note_maths"].mean()
```

Pandas : valeurs manquantes

Une valeur manquante (NaN) fausse certains calculs : on la repère, puis on supprime la ligne ou on remplace la valeur.

```python
print(df.isna().sum())
df2 = df.dropna()
print(len(df), "->", len(df2))
moy = df["note_info"].mean()
df["note_info"] = df["note_info"].fillna(moy)
print(round(moy, 2), df["note_info"].isna().sum())
```

Que choisir ?

Supprimer perd des données ; remplacer introduit une hypothèse. Le bon choix dépend du contexte : justifiez-le toujours dans votre notebook.

Résultat

```
nom           0
filiere       0
age           0
note_maths    1
note_info     2
dtype: int64
10 -> 7
13.75 0
```

À retenir

- isna().sum() compte les NaN par colonne
- dropna() supprime les lignes incomplètes (10 devient 7)
- fillna(moy) remplace par la moyenne : on garde les 10 lignes

NumPy ou Pandas : le comparatif

Pandas est construit au-dessus de NumPy : un DataFrame contient des tableaux NumPy, enrichis d'étiquettes.

Critère	NumPy (ndarray)	Pandas (DataFrame, Series)
Objet principal	ndarray : tableau n-dimensionnel	DataFrame (2D) et Series (1D)
Types	un seul type pour tout le tableau	un type par colonne
Accès aux éléments	par position : a[0, 1]	par position ou étiquette : loc, iloc
Dimensions	1D, 2D, 3D… n dimensions	1D (Series) ou 2D (DataFrame)
Valeurs manquantes	NaN seulement pour les flottants	isna(), fillna(), dropna()
Fichiers	bas niveau (np.loadtxt)	read_csv(), read_excel(), to_csv()
Usage typique	calcul numérique, algèbre, images	nettoyage, exploration, groupby
Poids	très léger et rapide	plus riche, un peu plus lourd

NumPy ou Pandas : voir la différence

Mêmes nombres, deux structures : positions seulement d'un côté, étiquettes de l'autre.

NumPy : ndarray

	0	1
0	14.5	16.0
1	17.0	14.0

Pas de noms : seulement 0, 1, 2…

Un type unique : float64

`arr[0, 1]  →  16.0`

Pandas : DataFrame

	maths	info
Amine	14.5	16.0
Sara	17.0	14.0

Noms de colonnes et étiquettes de lignes

Un type par colonne (nombres, texte, dates…)

`df.loc["Amine", "info"]  →  16.0`

En code : positions ou étiquettes

Même contenu dans les deux outils. Le calcul est identique, mais Pandas garde les noms dans le résultat.

Python

```python
import numpy as np
import pandas as pd

arr = np.array([[14.5, 16.0],
                [17.0, 14.0]])
df_s = pd.DataFrame(arr, columns=["maths", "info"],
                    index=["Amine", "Sara"])
print(arr[0, 1], df_s.loc["Amine", "info"])
print(arr.mean(axis=0))
print(df_s.mean())
```

Résultat

```
16.0 16.0
[15.75 15.  ]
maths    15.75
info     15.00
dtype: float64
```

À lire

Le résultat NumPy [15.75 15.] ne dit pas quelle moyenne correspond à quelle matière. Le résultat Pandas le dit.

À retenir

- arr.mean(axis=0) renvoie un tableau sans noms
- df_s.mean() renvoie une Series : maths et info sont nommés
- Par défaut, Pandas calcule par colonne

En code : types mélangés

Un tableau NumPy n'a qu'un seul type : si on mélange texte et nombres, tout devient du texte. Pandas garde un type par colonne.

Python

```python
mix = np.array([["Amine", 20, 14.5]])
print(mix.dtype)

df_mix = pd.DataFrame({"nom": ["Amine"],
                       "age": [20],
                       "note": [14.5]})
print(df_mix.dtypes)
print(type(df_mix.to_numpy()))
```

Résultat

```
<U32
nom          str
age        int64
note     float64
dtype: object
<class 'numpy.ndarray'>
```

Conséquence

Dans mix, on ne peut plus calculer 14.5 * 2 : c'est devenu le texte "14.5". Pour un fichier avec noms, âges et notes, Pandas s'impose.

À retenir

- <U32 : toutes les valeurs sont devenues du texte
- Pandas conserve str, int64 et float64
- to_numpy() reconvertit un DataFrame en ndarray

Quand utiliser quoi ?

Règle simple : des nombres purs et du calcul → NumPy ; un tableau de données avec des noms et des types variés → Pandas.

Choisir NumPy pour

- Calcul matriciel et opérations vectorisées
- Images, signaux, mesures de même type
- Entrées d'un algorithme de Machine Learning
- Performance et faible consommation mémoire

Choisir Pandas pour

- Lire et écrire des CSV ou Excel
- Colonnes de types différents (texte, nombres, dates)
- Nettoyer : valeurs manquantes, doublons
- Filtrer, trier, regrouper (groupby)

```
arr = df.to_numpy()          # Pandas vers NumPy
df = pd.DataFrame(arr, columns=cols)   # NumPy vers Pandas
```

Les deux se combinent : on prépare les données avec Pandas, on calcule avec NumPy.

Exercices de la partie 2

À réaliser dans votre notebook, cellule par cellule.

1. **NumPy : tableau 3×4**
   Créer np.arange(1, 13).reshape(3, 4), extraire la 2e colonne et la dernière ligne.
2. **NumPy : masque et axis**
   Garder les valeurs supérieures à 6 ; calculer la somme par colonne puis par ligne.
3. **NumPy : centrer-réduire**
   Calculer (a - a.mean()) / a.std() et vérifier que la moyenne obtenue est proche de 0.
4. **Pandas : explorer etudiants.csv**
   Charger le fichier, afficher shape, dtypes, head() et describe().
5. **Pandas : filtrer et trier**
   Afficher nom et note_maths des étudiants de Physique, triés par note décroissante.
6. **Pandas : nouvelle colonne**
   Créer moyenne = (note_maths + note_info) / 2 puis calculer la moyenne par filière avec groupby.
7. **Pandas : valeurs manquantes**
   Compter les NaN, puis les remplacer par la moyenne de leur colonne. Justifier ce choix.